Introduction to Theoretical Computer Science

CS2490 - Session 23: Decision algorithms for regular languages

Pierre Cagne

Fall 2026

October 12, 2026

Last time

A regular grammar G is one for which all production rules are of the form:

- $A \to wB$, or
- $A \to w$

where A, B are variables and w is a terminal word.

Last time

A regular grammar G is one for which all production rules are of the form:

- $A \to wB$, or
- $A \to w$

where A, B are variables and w is a terminal word.

We have seen how to:

Last time

A regular grammar G is one for which all production rules are of the form:

- $A \to wB$, or
- $A \to w$

where A, B are variables and w is a terminal word.

We have seen how to:

- transform a regular grammar into a NFA, and
  $\to$ each state of the NFA is either a variable or a intermediary state.

Last time

A regular grammar G is one for which all production rules are of the form:

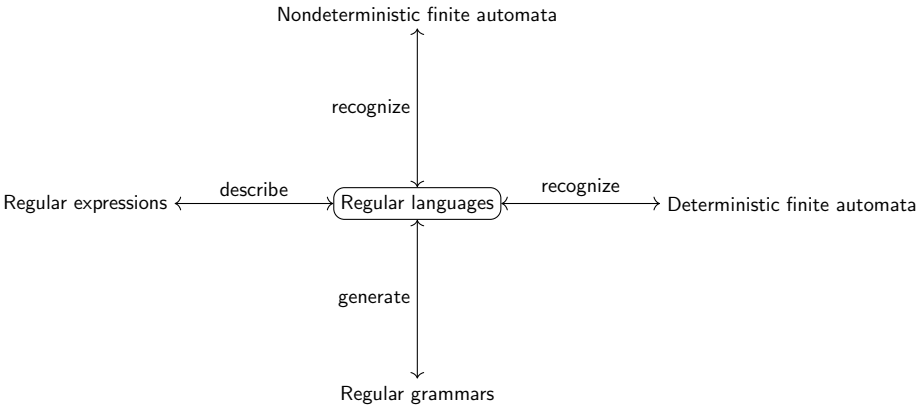
- $A \to wB$, or
- $A \to w$

where A, B are variables and w is a terminal word.

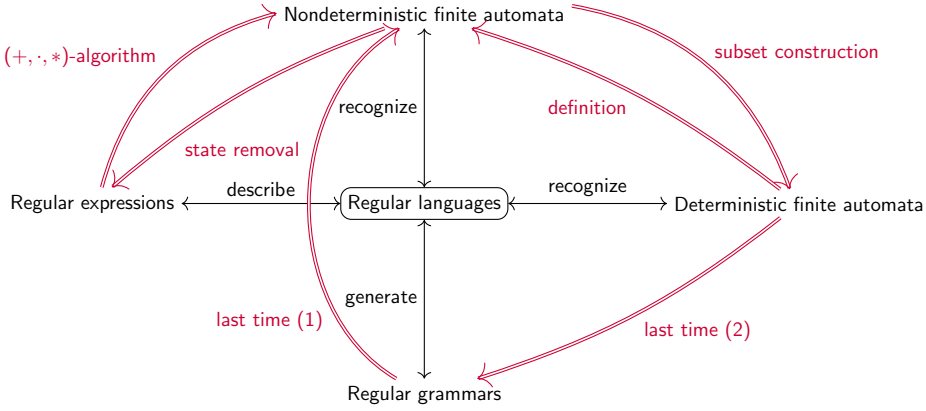
We have seen how to:

- transform a regular grammar into a NFA, and
  $\to$ each state of the NFA is either a variable or a intermediary state.
- transform a DFA into a regular grammar.
  $\to$ variables of the grammar are exactly the states of the DFA.

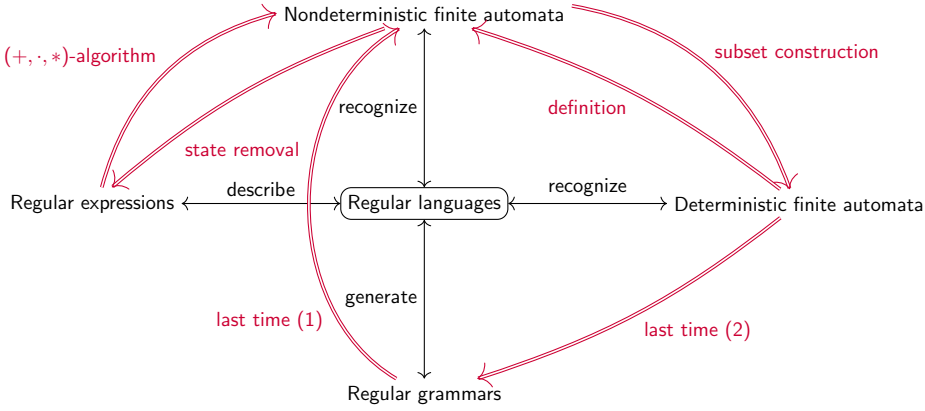
Summary


Nondeterministic finite automata
recognize
Regular expressions
describe
Regular languages
recognize
Deterministic finite automata
generate
Regular grammars

Summary


Nondeterministic finite automata
$(+, \cdot, *)$-algorithm
subset construction
recognize
definition
state removal
Regular expressions
describe
Regular languages
recognize
Deterministic finite automata
generate
last time (1)
last time (2)
Regular grammars

Summary



Note: All the $\Rightarrow$s are *algorithmic* transformations.

Standard representations

It gives 4 ways to represent a regular language algorithmically.

Standard representations

It gives 4 ways to represent a regular language algorithmically.

(We will see later a rigorous definition of "algorithmically". For now, think "that can be fed to a computer program".)

Standard representations

It gives 4 ways to represent a regular language algorithmically.

(We will see later a rigorous definition of "algorithmically". For now, think "that can be fed to a computer program".)

When a regular language is given in any of these forms, we say it is given in a standard representation.

Standard representations

It gives 4 ways to represent a regular language algorithmically.

(We will see later a rigorous definition of "algorithmically". For now, think "that can be fed to a computer program".)

When a regular language is given in any of these forms, we say it is given in a standard representation.

We are interested in questions of the form:
"Given a regular language in a standard representation, is there an algorithm that decides whether . . .?"

Membership testing

Decision problem

Given a regular language L (in standard representation) and a word w, is there an algorithm that decides whether $w \in L$?

Membership testing

Decision problem

Given a regular language L (in standard representation) and a word w, is there an algorithm that decides whether $w \in L$?

In other words, can you write a Java program returning a boolean answering this question?

Membership testing

Decision problem

Given a regular language L (in standard representation) and a word w, is there an algorithm that decides whether $w \in L$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Membership testing

Decision problem

Given a regular language L (in standard representation) and a word w, is there an algorithm that decides whether $w \in L$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation:

Membership testing

Decision problem

Given a regular language L (in standard representation) and a word w, is there an algorithm that decides whether $w \in L$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Transform the representation of L into a DFA and simulate the DFA on the word w:

Membership testing

Decision problem

Given a regular language L (in standard representation) and a word w, is there an algorithm that decides whether $w \in L$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Transform the representation of L into a DFA and simulate the DFA on the word w:

- Start at the initial state.

Membership testing

Decision problem

Given a regular language L (in standard representation) and a word w, is there an algorithm that decides whether $w \in L$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Transform the representation of L into a DFA and simulate the DFA on the word w:

- Start at the initial state.
- For each letter of w, follow the corresponding transition.

Membership testing

Decision problem

Given a regular language L (in standard representation) and a word w, is there an algorithm that decides whether $w \in L$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Transform the representation of L into a DFA and simulate the DFA on the word w:

- Start at the initial state.
- For each letter of w, follow the corresponding transition.
- If you end in a final state, then $w \in L$, otherwise $w \notin L$.

Membership testing (cont'd)

Note that we only care if such an algorithm exists or not.

Membership testing (cont'd)

Note that we only care if such an algorithm exists or not.

We have no interest in how efficient it is.

Membership testing (cont'd)

Note that we only care if such an algorithm exists or not.

We have no interest in how efficient it is (for now).

Emptiness testing

Decision problem

Given a regular language L (in standard representation), is there an algorithm that decides whether $L = \emptyset$?

Emptiness testing

Decision problem

Given a regular language L (in standard representation), is there an algorithm that decides whether $L = \emptyset$?

In other words, can you write a Java program returning a boolean answering this question?

Emptiness testing

Decision problem

Given a regular language L (in standard representation), is there an algorithm that decides whether $L = \emptyset$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Emptiness testing

Decision problem

Given a regular language L (in standard representation), is there an algorithm that decides whether $L = \emptyset$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation:

Emptiness testing

Decision problem

Given a regular language L (in standard representation), is there an algorithm that decides whether $L = \emptyset$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Transform the representation of L into a DFA. Check if there is a path from the start state to any final state using a BFS of the graph of states:

Emptiness testing

Decision problem

Given a regular language L (in standard representation), is there an algorithm that decides whether $L = \emptyset$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Transform the representation of L into a DFA. Check if there is a path from the start state to any final state using a BFS of the graph of states:

- Initially, mark the start state as visited and add it to a queue.

Emptiness testing

Decision problem

Given a regular language L (in standard representation), is there an algorithm that decides whether $L = \emptyset$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Transform the representation of L into a DFA. Check if there is a path from the start state to any final state using a BFS of the graph of states:

- Initially, mark the start state as visited and add it to a queue.
- While the queue is not empty:

Emptiness testing

Decision problem

Given a regular language L (in standard representation), is there an algorithm that decides whether $L = \emptyset$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Transform the representation of L into a DFA. Check if there is a path from the start state to any final state using a BFS of the graph of states:

- Initially, mark the start state as visited and add it to a queue.
- While the queue is not empty:
  - Dequeue a state s.

Emptiness testing

Decision problem

Given a regular language L (in standard representation), is there an algorithm that decides whether $L = \emptyset$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Transform the representation of L into a DFA. Check if there is a path from the start state to any final state using a BFS of the graph of states:

- Initially, mark the start state as visited and add it to a queue.
- While the queue is not empty:
  - Dequeue a state s.
  - If s is a final state, return false (the language is not empty).

Emptiness testing

Decision problem

Given a regular language L (in standard representation), is there an algorithm that decides whether $L = \emptyset$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Transform the representation of L into a DFA. Check if there is a path from the start state to any final state using a BFS of the graph of states:

- Initially, mark the start state as visited and add it to a queue.
- While the queue is not empty:
  - Dequeue a state s.
  - If s is a final state, return false (the language is not empty).
  - Otherwise, enqueue all unvisited states reachable from s and mark them as visited.

Emptiness testing

Decision problem

Given a regular language L (in standard representation), is there an algorithm that decides whether $L = \emptyset$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Transform the representation of L into a DFA. Check if there is a path from the start state to any final state using a BFS of the graph of states:

- Initially, mark the start state as visited and add it to a queue.
- While the queue is not empty:
  - Dequeue a state s.
  - If s is a final state, return false (the language is not empty).
  - Otherwise, enqueue all unvisited states reachable from s and mark them as visited.
- If the queue is empty and no final state was reached, return true (the language is empty).

Equivalence testing

Decision problem

Given two regular languages L_1 and L_2 (in standard representations), is there an algorithm that decides whether $L_1 = L_2$?

Equivalence testing

Decision problem

Given two regular languages L_1 and L_2 (in standard representations), is there an algorithm that decides whether $L_1 = L_2$?

In other words, can you write a Java program returning a boolean answering this question?

Equivalence testing

Decision problem

Given two regular languages L_1 and L_2 (in standard representations), is there an algorithm that decides whether $L_1 = L_2$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Equivalence testing

Decision problem

Given two regular languages L_1 and L_2 (in standard representations), is there an algorithm that decides whether $L_1 = L_2$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation:

Equivalence testing

Decision problem

Given two regular languages L_1 and L_2 (in standard representations), is there an algorithm that decides whether $L_1 = L_2$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Notice that $L_1 = L_2$ if and only if both $L_1 \subseteq L_2$ and $L_2 \subseteq L_1$. In other words, $L_1 = L_2$ if and only if $L_1 \cap \overline{L_2} = \emptyset$ and $L_2 \cap \overline{L_1} = \emptyset$.

Equivalence testing

Decision problem

Given two regular languages L_1 and L_2 (in standard representations), is there an algorithm that decides whether $L_1 = L_2$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Notice that $L_1 = L_2$ if and only if both $L_1 \subseteq L_2$ and $L_2 \subseteq L_1$. In other words, $L_1 = L_2$ if and only if $L_1 \cap \overline{L_2} = \emptyset$ and $L_2 \cap \overline{L_1} = \emptyset$.
Transform the representations of L_1 and L_2 into DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$.

Equivalence testing

Decision problem

Given two regular languages L_1 and L_2 (in standard representations), is there an algorithm that decides whether $L_1 = L_2$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Notice that $L_1 = L_2$ if and only if both $L_1 \subseteq L_2$ and $L_2 \subseteq L_1$. In other words, $L_1 = L_2$ if and only if $L_1 \cap \overline{L_2} = \emptyset$ and $L_2 \cap \overline{L_1} = \emptyset$.
Transform the representations of L_1 and L_2 into DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$.

- Construct DFAs recognizing $L_1 \cap \overline{L_2}$ and $L_2 \cap \overline{L_1}$ using the complementation and product algorithms.

Equivalence testing

Decision problem

Given two regular languages L_1 and L_2 (in standard representations), is there an algorithm that decides whether $L_1 = L_2$?

In other words, can you write a Java program returning a boolean answering this question?

Answer: Yes.

Implementation: Notice that $L_1 = L_2$ if and only if both $L_1 \subseteq L_2$ and $L_2 \subseteq L_1$. In other words, $L_1 = L_2$ if and only if $L_1 \cap \overline{L_2} = \emptyset$ and $L_2 \cap \overline{L_1} = \emptyset$.
Transform the representations of L_1 and L_2 into DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$.

- Construct DFAs recognizing $L_1 \cap \overline{L_2}$ and $L_2 \cap \overline{L_1}$ using the complementation and product algorithms.
- Use the emptiness testing algorithm to check if both languages are empty.